Week 1 Student Notes

HPC Introduction and Architectures

Course: Operating Systems 4 (HPC)
Module: EIOSY4A
Topic: HPC Introduction, Architectures & Parallelism
Reading Time: ~20 Minutes

Learning Outcomes

By the end of this week, you should be able to:

1. **Critique** the definition of High-Performance Computing relative to standard IT infrastructure in complex environments.
2. **Evaluate** the role and impact of the South African HPC ecosystem (CHPC, DIRISA, SANReN) on regional research.
3. **Analyze** cluster performance using FLOPS and interpret top-tier benchmarking standards (Top500).
4. **Contrast and evaluate** the Von Neumann and Harvard architectures in the context of modern supercomputing.
5. **Formulate** strategies to mitigate the Memory Wall problem using advanced cache hierarchies.
6. **Classify** parallel computing systems critically using Flynn's Taxonomy.
7. **Apply and analyze** Amdahl's Law and Gustafson's Law to predict parallel speedup bounds.

Week Goal

Welcome to HPC 2026! This week we're cutting through the buzzwords and getting down to what "High Performance Computing" actually means in practice. We'll look at how modern processors are built from the ground up, and — more importantly — we'll learn the fundamental laws that tell us exactly how much speed we can squeeze out of parallel hardware.

Assessment Alignment: The weekly quiz and practical assignments for this module specifically evaluate the Learning Outcomes listed below, verifying your ability to operate at an NQF Level 7 standard of analysis and synthesis.

1. Introduction to HPC

Let's get one thing straight from the start: High Performance Computing isn't just about having "fast computers." It's the science and engineering of pooling computing power together — hundreds or thousands of machines working as one — to tackle problems that no single desktop or server could ever handle alone (Sterling et al., 2018).

1.1 The Need for Speed (and Scale)

In everyday IT, we tend to think about throughput: how many web requests can a server handle per second? HPC flips the question. Here, what matters is **Speedup** — how much faster can we solve a *single, massive* problem by splitting the work across multiple processors?

The real world makes the case better than any textbook. Training large AI models like GPT takes thousands of GPUs crunching through petabytes of text in parallel. In medicine, assembling a complete human genome and hunting for cancer mutations requires raw computational muscle that a single workstation simply can't provide. On the Johannesburg Stock Exchange (JSE), high-frequency trading algorithms analyse market data in microseconds — fall behind, and you lose money. Formula 1 teams use Computational Fluid Dynamics (CFD) to simulate airflow over car bodies long before they ever hit a wind tunnel. And right here at home, the South African Weather Service (SAWS) runs daily weather simulations — solving Navier-Stokes equations across a grid covering the entire country — to issue the storm warnings we all rely on.

1.2 Local Context: The South African Cyberinfrastructure Ecosystem

You might think supercomputing is a first-world luxury, but South Africa actually hosts the most advanced research computing infrastructure on the continent. It's all coordinated under the ***National Integrated Cyber Infrastructure System (NICIS)***, which stands on three pillars (CHPC, 2024).

The first is the ***Centre for High Performance Computing (CHPC)*** in Cape Town. Their flagship system, *Lengau*, is one of the fastest supercomputers in the Southern Hemisphere and it's available to any SA university or research council — including ours. Then there's ***DIRISA (Data Intensive Research Infrastructure of South Africa)***, which handles the long-term storage and curation of massive research datasets, especially from astronomy projects. Tying it all together is ***SANReN (South African National Research Network)***, the high-speed fibre backbone that lets researchers move terabytes of data between campuses and national facilities without bottlenecks (SANReN, 2024).

This isn't just infrastructure for the sake of it — real projects depend on it every day. The ***Square Kilometre Array (SKA)*** radio telescope in the Karoo produces a firehose of cosmic data that demands Exascale processing. The ***South African Weather Service (SAWS)*** runs numerical weather prediction models on dedicated HPC clusters. And during the COVID-19 pandemic, the ***National Institute for Communicable Diseases (NICD)*** leaned heavily on these resources for genomic sequencing, tracking variants in near real-time (NICD, 2024).

1.3 Measuring Performance: The FLOP

In standard IT, we might measure a server's performance in "Instructions Per Second." But in scientific computing — where the workload is overwhelmingly mathematical — the yardstick is the ***Floating-Point Operation (FLOP)***. Think of it as a single multiplication or addition of two decimal numbers, like 2.5 * 3.14. The laptop you're sitting with right now likely operates in the ***GigaFLOPS (GFLOPS)*** range — roughly 10^9 operations per second. A beefy GPU can hit ***TeraFLOPS (TFLOPS)*** territory (10^{12}). The supercomputers we're studying this semester crunch numbers at the ***PetaFLOPS (PFLOPS)*** scale (10^{15}), and the newest machines have smashed through the ***ExaFLOPS (EFLOPS)*** barrier (10^{18}) — that's a quintillion calculations every single second.

1.4 The Top500 List

Every six months, the supercomputing community publishes the **Top500** list — a global ranking of the 500 most powerful non-distributed computer systems (Top500.org, 2024). The ranking is based on the ***HPL (High Performance Linpack)*** benchmark, which solves a dense system of

linear equations. Here's the exciting part: we'll actually run this exact benchmark on our own cluster in Week 7.

2. System Architecture

Before we start bolting computers together into a cluster, we need to understand what's going on inside a single machine. Nearly every modern computer follows the ***Von Neumann Architecture***, a design that dates all the way back to 1945 (Sterling et al., 2018).

2.1 The Von Neumann Model

At its heart, the Von Neumann model is a "stored-program" computer — meaning instructions and data live together in the same read-write memory (RAM). The system has three main parts: the Central Processing Unit (CPU), which houses both the Control Unit and the Arithmetic Logic Unit (ALU); Main Memory; and the I/O system. The crucial design decision? Everything shares a single bus.

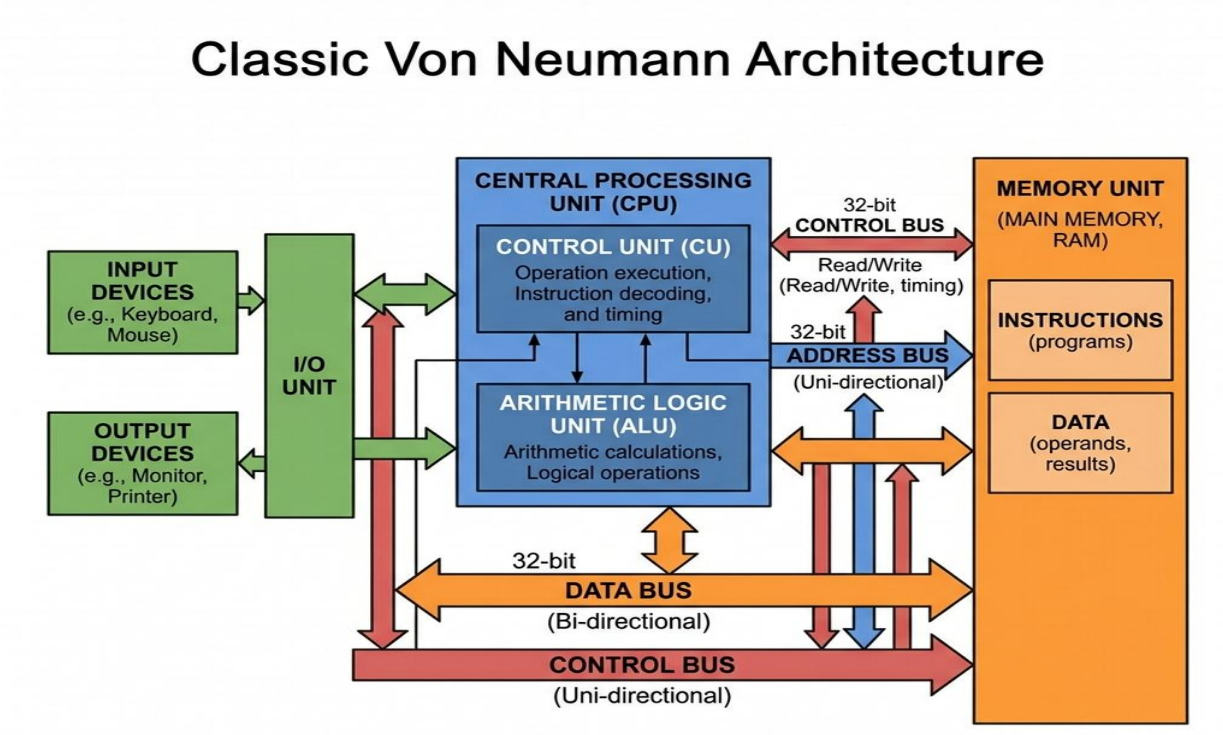


Figure 1: The Von Neumann Architecture — a shared bus connects CPU, Memory, and I/O. Instructions and data compete for the same pathway, which creates a bottleneck.

That shared pathway is exactly where the trouble starts. It creates what we call the ***Von Neumann Bottleneck***. The CPU can crunch numbers far faster than the memory bus can deliver them, so the processor ends up sitting idle for hundreds of clock cycles just waiting for data to arrive. Think of it as a race car stuck in traffic — the engine is ready to go, but the road can't keep up. This "starvation" of the functional units remains the single biggest architectural headache in modern computing (Sterling et al., 2018).

2.2 The Harvard Architecture

The Harvard Architecture takes a different approach: it gives instructions and data their own *separate* storage and signal pathways. That means the processor can fetch the next instruction and read data at the same time, essentially doubling the bandwidth. In its pure form, you'll mostly find Harvard architectures in Digital Signal Processors (DSPs) and embedded microcontrollers. But here's the interesting bit: modern general-purpose CPUs — your Intel Core i9, your AMD EPYC server chips — actually use a ***Modified Harvard Architecture*** internally. They keep separate L1 caches for Instructions (L1i) and Data (L1d) to get the bandwidth benefits, while still presenting a single, unified memory address space to the programmer. It's the best of both worlds.

3. The Memory Wall and Caching

3.1 The Memory Wall

Here's a problem that has haunted computer architects for decades. Over the last forty years, processor speeds have improved exponentially — riding Moore's Law upward — while DRAM memory latency has crept along at a much slower, roughly linear pace. The gap between these two curves keeps widening, and that gap has a name: the ***Memory Wall***.

To put this in concrete terms: a modern CPU core running at 4GHz can execute a simple instruction in about 0.25 nanoseconds. But fetching a piece of data from main memory (DRAM) takes roughly 100 nanoseconds. That means a single cache miss — one unlucky memory fetch — leaves the processor sitting idle for about 400 clock cycles. For high-performance applications, that kind of waste is devastating.

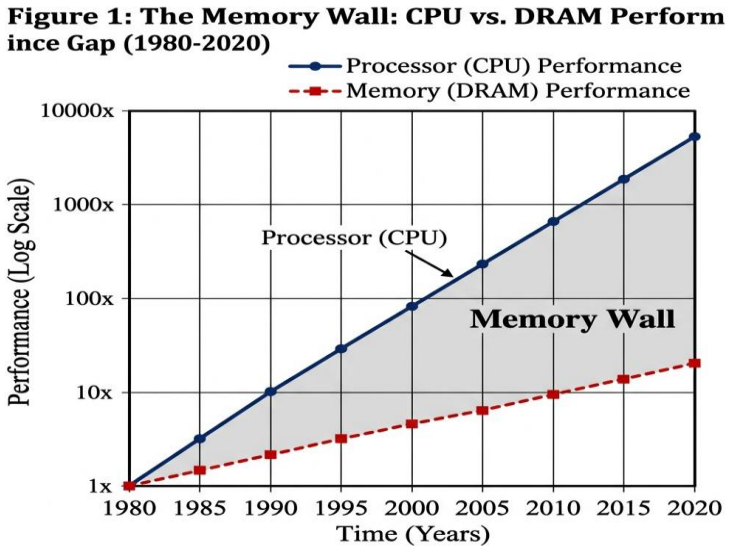


Figure 2: The Memory Wall — the widening gap between CPU speed (exponential growth) and Memory speed (linear growth) is the reason we need complex caching strategies.

3.2 The Solution: Cache Hierarchy

So how do we deal with a CPU that can think a hundred times faster than memory can deliver? The answer is caching. Architects insert layers of smaller, faster, and more expensive Static RAM (SRAM) between the CPU and DRAM. This ***Cache Hierarchy*** works because of a simple observation called ***Locality of Reference*** — programs tend to access the same data (or nearby data) repeatedly. There are two flavours: ***Temporal Locality*** means that if you just used a variable (say, a loop counter), you'll probably use it again very soon. ***Spatial Locality*** means that if you accessed Matrix[i][0], you'll likely want Matrix[i][1] next.

Modern HPC processors like the AMD EPYC take this idea and run with it. The ***L1 Cache*** is tiny (32KB–64KB) but blazing fast — it runs at the core's full clock speed and is split into separate Instruction and Data halves. The ***L2 Cache*** sits one step back (512KB–1MB), usually private to a single core. Then there's the ***L3 Cache***, which is much larger (32MB up to 1GB with AMD's 3D V-Cache technology) and shared across all cores on the chip. L3 is essentially the last line of defence before the CPU has to go all the way out to slow DRAM.

4. Processor Architecture (Sterling Chapter 3)

When we're trying to reason about parallel algorithms, it helps to start with an idealised model. The ***PRAM (Parallel Random Access Machine)*** gives us exactly that — an imaginary machine where multiple processors all share a single pool of memory. Real clusters are messier (they use distributed memory connected by networks), but the PRAM model lets us think clearly about algorithmic complexity before we worry about hardware headaches.

At its core, the whole point of HPC processor design is simple: execute more floating-point operations per second. For years, the industry achieved this by cranking up the clock speed — a strategy called Frequency Scaling. But around 2005, physics hit back. Pushing frequencies higher produced more heat than conventional cooling could handle — a barrier known as the ***Power Wall***. Since then, the focus has shifted from making individual cores faster to making them smarter through parallelism (Sterling et al., 2018).

4.1 Pipelining and Superscalar Execution

How do you get more work done without turning up the clock? One answer is ***Instruction Pipelining***. Think of it like a car assembly line: instead of one worker building an entire car from scratch before starting the next one, you break the process into stages — Fetch, Decode, Execute, Write-back — and overlap them. While one instruction is being executed, the next one is already being decoded, and a third is being fetched. The result? You finish one instruction every clock cycle instead of every four, effectively quadrupling throughput. Modern ***Superscalar*** processors take it further by duplicating the pipeline itself, so they can issue multiple independent instructions — what we call Instruction Level Parallelism (ILP) — in a single cycle.

4.2 Flynn's Taxonomy

Now that we've seen how individual cores squeeze out performance, let's zoom out. ***Flynn's Taxonomy*** gives us a clean framework for classifying parallel architectures based on how many instruction streams and data streams they handle at once (Sterling et al., 2018).

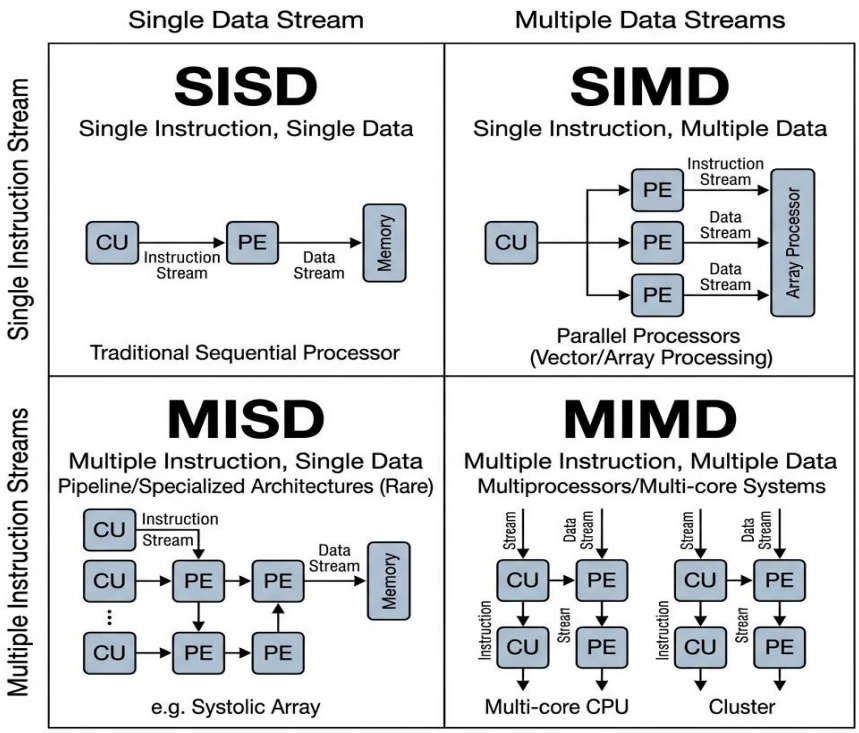


Figure 3: Flynn's Taxonomy — SISD (traditional sequential), SIMD (vector processing / GPU), and MIMD (cluster computing).

4.2.1 SISD (Single Instruction, Single Data)

This is the classic sequential model — one processor, one instruction at a time, one piece of data. It's what you'd find in early computers and basic microcontrollers. There's no parallelism at all; everything happens strictly one step after the other. An ADD instruction finishes before a LOAD can begin.

4.2.2 MIMD (Multiple Instruction, Multiple Data)

MIMD is the model that describes an HPC **Cluster** as a whole. You have multiple autonomous processors, each running its own independent program on its own data. Node 1 might be simulating airflow over a wing, while Node 2 is calculating engine heat dissipation — completely different tasks, running simultaneously, coordinated by a management layer. This is the "big picture" parallelism of supercomputing.

4.2.3 SIMD (Single Instruction, Multiple Data)

While MIMD handles the cluster level, **SIMD** is where the magic happens inside each *individual core*. The idea is beautifully simple: one instruction does the same thing to many data points at once. This is called **Vectorization**. Instead of writing a loop that adds two numbers at a time (C = A + B), a SIMD instruction loads a whole "vector" of data — say, 8 double-precision numbers — into a wide 512-bit register (like Intel's AVX-512) and adds them all in a single clock cycle. That's an 8x speedup for number-heavy workloads like matrix multiplication, which is the bread and butter of almost every scientific simulation.

4.3 Multicore and Amdahl's Law

Since individual cores hit the Power Wall and couldn't run any faster, manufacturers started putting more cores on a single chip. But here's the catch — more cores doesn't automatically mean proportionally faster programs. ***Amdahl's Law*** defines the hard theoretical limit on speedup (S), and it all comes down to the fraction of your program that *can't* be parallelised ($1 - p$).

$$S = \frac{1}{(1-p) + \frac{p}{s}}$$

The maths is sobering. Even with infinite processors ($s \to \infty$), the speedup is capped by the serial portion. If just 10% of your code is inherently sequential — say, reading an input file or writing final results — then no matter how many cores you throw at the problem, you will never run more than 10x faster. That's a reality check every HPC programmer has to face.

4.4 Gustafson's Law: The HPC Justification

Amdahl's Law paints a pretty grim picture, but it makes one key assumption: that the problem size stays fixed (this is called **Strong Scaling**). ***Gustafson's Law*** offers a much more hopeful perspective. It says: as we get more processors, we won't just solve the same problem faster — we'll solve *bigger* problems. This is **Weak Scaling**.

$$S_p = p + (1-p)s$$

Here, s is the serial fraction of execution time. The key insight is that the serial part doesn't bottleneck you nearly as much when the parallel portion of the workload grows with the machine. And this is precisely the justification for building Exascale systems: we're not just trying to run yesterday's weather model in fewer seconds — we're trying to run tomorrow's global climate model at a resolution that was never possible before.

5. Career Pathways in HPC

So where does all this knowledge take you professionally? The most direct path is becoming an ***HPC Systems Engineer*** — designing and tuning the massive supercomputers at national labs or research universities. But the skills you're learning here transfer directly to industry roles too. A ***Cloud Solutions Architect*** *or* ***Site Reliability Engineer (SRE)*** at a company like Amazon or Takealot needs the exact same understanding of latency, throughput, and resource scheduling — they're just managing Kubernetes clusters instead of Beowulf ones. There's also a rapidly growing role called ***Research Software Engineer (RSE)***, which bridges the gap between scientists who write messy code and the hardware that has to run it. RSEs optimise algorithmic code for parallel hardware, and they're in huge demand in Fintech and AI development.

6. Summary and Next Steps

Let's recap what we covered this week. We've drawn a clear line between everyday computing (focused on latency) and HPC (focused on throughput and speedup). We've explored the landscape of parallel architectures — from the basic SISD model all the way up to MIMD clusters and SIMD vectorization. And we've grounded all of this in a South African context, understanding that the **CHPC** isn't some distant concept — it's a national resource that researchers at our university can actually use.

Next Week: We move from abstract architectures to concrete system design — **System Design and Memory**.

Checklist for Next Week

☐ Can you explain the difference between **Throughput** and **Latency**?

☐ Do you know which classification (SISD, SIMD, MISD, MIMD) a modern GPU falls into?

☐ Have you identified the Top 500 list and where South Africa stands?

☐ If a CPU has AVX-512 (can process 8 doubles at once), do you understand why vectorized code is faster than scalar code?

Resources

- **Centre for High Performance Computing (CHPC)**. (2024). *About the CHPC*. Available at: https://www.chpc.ac.za (Accessed: 22 December 2025).
- **National Institute for Communicable Diseases (NICD)**. (2024). *Genomic Surveillance*. Available at: https://www.nicd.ac.za (Accessed: 22 December 2025).
- **SANReN**. (2024). *South African National Research Network Overview*. Available at: https://www.sanren.ac.za (Accessed: 22 December 2025).
- **Sterling, T., Anderson, M. and Brodowicz, M. (2018)**. *High Performance Computing: Modern Systems and Practices*. Morgan Kaufmann.
- **Top500.org**. (2024). *TOP500 Supercomputer Sites*. Available at: https://www.top500.org (Accessed: 22 December 2025).
- **Trobec, R., Slivnik, B., Bulić, P. and Robič, B. (2018)**. *Introduction to Parallel Computing*. Springer.